Zero One Knapsack

$$ks(i, w') = \begin{cases} ks(i-1, w') & \text{if } w[i] > w' \\ \max\{ks(i-1, w'),\ P[i] + ks(i-1, w'-w[i])\} & \text{if } w[i] \le w' \end{cases}$$

Similar Problem Statements

* Subset Sum Problem — true or false
* Equal Sum Partition — sum of the set ÷ 2
* Count of Subsets that sums to a given sum — remove max and swap or with + in subset sum
* Minimum Subset Sum difference — the last row is what matters
* Number of Subsets with given difference — [sum + difference] ÷ 2
* Target Sum (sign assignment problem) — can be reduced to the previous problem

Unbounded Knapsack

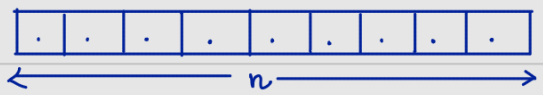
Solution Description : The basic approach to the problem remains the same as the 0/1 Knapsack Problem, with a slight modification that after taking the item i in the Knapsack, unlike 0/1 Knapsack algorithm, we will not increment the index i. As a result, we will always have the scope of taking item i in the Knapsack more than once.

Recurrance Relation :

$$\text{Knapsack}(i, w) = \begin{cases} \text{Knapsack}(i+1, w) & \text{if } wt[i] > w \\ \max\left\{ \begin{array}{l} \text{Knapsack}(i+1, w) \\ ct[i] + \text{Knapsack}(i, w - wt[i]) \end{array} \right\} & \text{otherwise} \end{cases}$$

Note : Unlike 0/1 Knapsack, we do not need a 2D Array for the DP version of the Solution.

Dynamic Programming :

$M =$ 

$n =$ capacity

$$M[w] = \text{for } i \in [1, n] \left\{ \max\left(M[w],\ M[w - wt[i]] + pf[i] \right) \right\}$$
$$\forall\ w \in [1, n]$$

2D Realization of Unbounded Knapsack :

The 2D realization of the problem can be given with the following relation :

$$uks(i, w') = \begin{cases} uks(i-1, w') & \text{if } w[i] > w' \\ \max\{ uks(i-1, w'),\ P[i] + uks(i, w' - w[i]) & \text{if } w[i] \leq w' \end{cases}$$

Similar Problem Statements :

* Rod Cutting Problem
* Coin Exchange Problem : Number of ways

  Unbounded Knapsack version of count of subsets with a given sum
* Coin Exchange Problem : Min no. of coins

Longest Common Subsequence

$$Lcs(i,j) = \begin{cases} 1 + Lcs(i-1, j-1) & \text{if } A[i] = B[j] \\ \max\{Lcs(i-1, j), Lcs(i, j-1)\} & \text{if } A[i] \neq B[j] \end{cases}$$

Similar Problem Statements :

* Printing Longest Common Subsequence
* Longest Common Substring $M[i,j] = 0$ if $A[i] \neq B[j]$
* Shortest Common Supersequence $SCS = m + n - LCS$
* Min. deletion and insertion to make A to B $m - LCS$
* Longest Palindromic Subsequence LPS = LCS of A and rev(A)
* Min. deletion/insertion to make A palindrome $m - LPS$
* Printing Shortest Common Supersequence
* Longest Repeating Subsequence $M[i,j] = 1 + M[i-1, j-1]$ if $A[i] = A[j]$ and $i \neq j$
* Edit distance between two strings

  Problem Statement : Return the count of insertion, deletion and change required to convert string A to string B.

  At any instance, if $A[i]$ matches $B[j]$, then $M[i,j] = M[i-1, j-1]$

  otherwise, $M[i,j]$ = min of { insertion, deletion and replace } + 1

$$M[i,j] = \begin{cases} M[i-1, j-1], & \text{if } A[i] = B[j] \\ \min\{M[i-1, j], M[i, j-1], M[i-1, j-1]\} + 1, & \text{otherwise} \end{cases}$$

* Sequence Pattern Matching i.e. if A is a subsequence of B? $LCS = m$?

Matrix Chain Multiplication

$$MCM(i,j) = \begin{cases} \emptyset & \text{if } i=j \\ \min\{MCM(i,k) + MCM(k+1,j) + P[i-1]P[k]P[j]\} & \text{otherwise} \\ & \forall\, i \le k < j \end{cases}$$

Similar Problem Statements :

* Palindrome Partitioning
* Evaluate Boolean Expression to True
* Scrambled string
* Egg dropping problem.